CAT Assignment 1

ICS1502 & Introduction to Machine Learning

Name:

Vigneshwaran S

Reg no:

3122237001059

Mobile Phone Price Prediction

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score


data = pd.read_csv("mobile_price.csv")
data.head()
```

	Ratings	RAM	ROM	Mobile_Size	Primary_Cam	Selfi_Cam	Battery_Power	Price
0	4.3	4.0	128.0	6.00	48	13.0	4000	24999
1	3.4	6.0	64.0	4.50	48	12.0	4000	15999
2	4.3	4.0	4.0	4.50	64	16.0	4000	15000
3	4.4	6.0	64.0	6.40	48	15.0	3800	18999
4	4.5	6.0	128.0	6.18	35	15.0	3800	18999

```
print("Dataset Info:")
print(data.info())

print("\nMissing values:")
print(data.isnull().sum())

data = data.dropna()

data = data.select_dtypes(include=[np.number])

print("\nCleaned Dataset Shape:", data.shape)
data.describe()
```

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 807 entries, 0 to 806
Data columns (total 8 columns):
 #   Column         Non-Null Count  Dtype
```

```
X = data.drop('Price', axis=1).values
y = data['Price'].values.reshape(-1, 1)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

X_train_b = np.c_[np.ones((X_train.shape[0], 1)), X_train]
X_test_b = np.c_[np.ones((X_test.shape[0], 1)), X_test]

print("X_train_b shape:", X_train_b.shape)
print("y_train shape:", y_train.shape)
```

```
X_train_b shape: (645, 8)
y_train shape: (645, 1)
```

```
ROM             0
Mobile Size     0
Primary_Cam     0
Selfi_Cam       0
Battery Power   0
Price           0
dtype: int64
```

Closed-form

```
theta_closed = np.linalg.inv(X_train_b.T.dot(X_train_b)).dot(X_train_b.T).dot(y_train)
y_pred_closed = X_test_b.dot(theta_closed)

mse_closed = mean_squared_error(y_test, y_pred_closed)
r2_closed = r2_score(y_test, y_pred_closed)

print("Closed-form Linear Regression:")
print("MSE:", mse_closed)
print("R²:", r2_closed)
```

```
Closed-form Linear Regression:
MSE: 239357657.43148595
R²: 0.4332281397209144
```

50%	4.100000	6.000000	32.000000	4.770000	48.000000	8.000000	3000.000000	1699.000000
75%	4.400000	6.000000	64.000000	6.300000	48.000000	12.000000	3800.000000	18994.500000
max	4.800000	12.000000	256.000000	44.000000	64.000000	23.000000	6000.000000	153000.000000

Gradient Descent

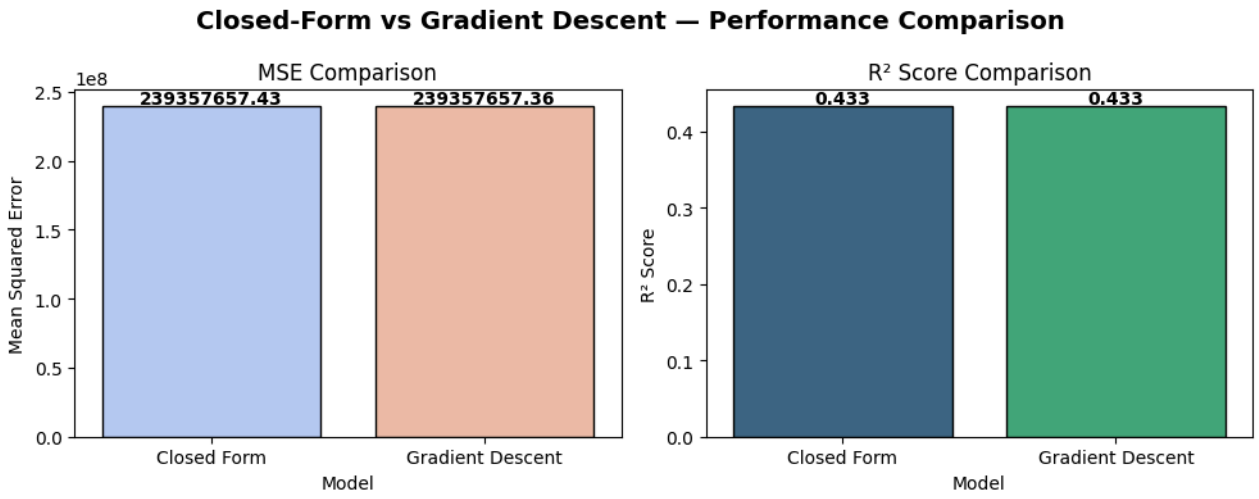
```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

X_train_b = np.c_[np.ones((X_train.shape[0], 1)), X_train]
X_test_b = np.c_[np.ones((X_test.shape[0], 1)), X_test]

def gradient_descent(X, y, lr=0.01, n_iter=2000):
    m, n = X.shape
    theta = np.zeros((n, 1))
    for i in range(n_iter):
        gradients = (2/m) * X.T.dot(X.dot(theta) - y)
        theta -= lr * gradients
    return theta

theta_gd = gradient_descent(X_train_b, y_train, lr=0.01, n_iter=2000)
y_pred_gd = X_test_b.dot(theta_gd)

mse_gd = mean_squared_error(y_test, y_pred_gd)
r2_gd = r2_score(y_test, y_pred_gd)

print("Gradient Descent Linear Regression:")
print("MSE:", mse_gd)
print("R²:", r2_gd)
```

```
Gradient Descent Linear Regression:
MSE: 239357657.3630127
R²: 0.4332281398830514
```

```
import matplotlib.pyplot as plt
import seaborn as sns
```

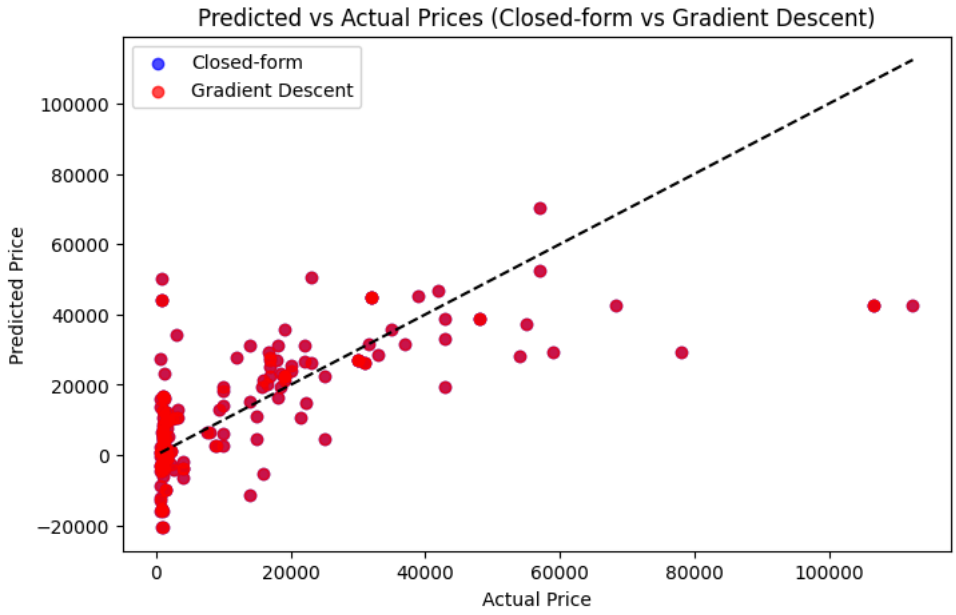
```
models = ['Closed Form', 'Gradient Descent']

mse_values = [mse_closed, mse_gd]
r2_values = [r2_closed, r2_gd]

plt.figure(figsize=(10,4))

plt.subplot(1, 2, 1)
sns.barplot(x=models, y=mse_values, palette='coolwarm', edgecolor='black')
plt.title('MSE Comparison')
plt.ylabel('Mean Squared Error')
plt.xlabel('Model')
for i, v in enumerate(mse_values):
    plt.text(i, v + 0.01 * max(mse_values), f"{v:.2f}", ha='center', fontweight='bold')

plt.subplot(1, 2, 2)
sns.barplot(x=models, y=r2_values, palette='viridis', edgecolor='black')
plt.title('R² Score Comparison')
plt.ylabel('R² Score')
plt.xlabel('Model')
for i, v in enumerate(r2_values):
    plt.text(i, v + 0.01 * max(r2_values), f"{v:.3f}", ha='center', fontweight='bold')

plt.suptitle('Closed-Form vs Gradient Descent — Performance Comparison', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```

```
C:\Users\Asus Vivobook\AppData\Local\Temp\ipykernel_13696\1926837655.py:18: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

  sns.barplot(x=models, y=mse_values, palette='coolwarm', edgecolor='black')
C:\Users\Asus Vivobook\AppData\Local\Temp\ipykernel_13696\1926837655.py:27: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

  sns.barplot(x=models, y=r2_values, palette='viridis', edgecolor='black')
```



Plot predicted vs. actual values on test data

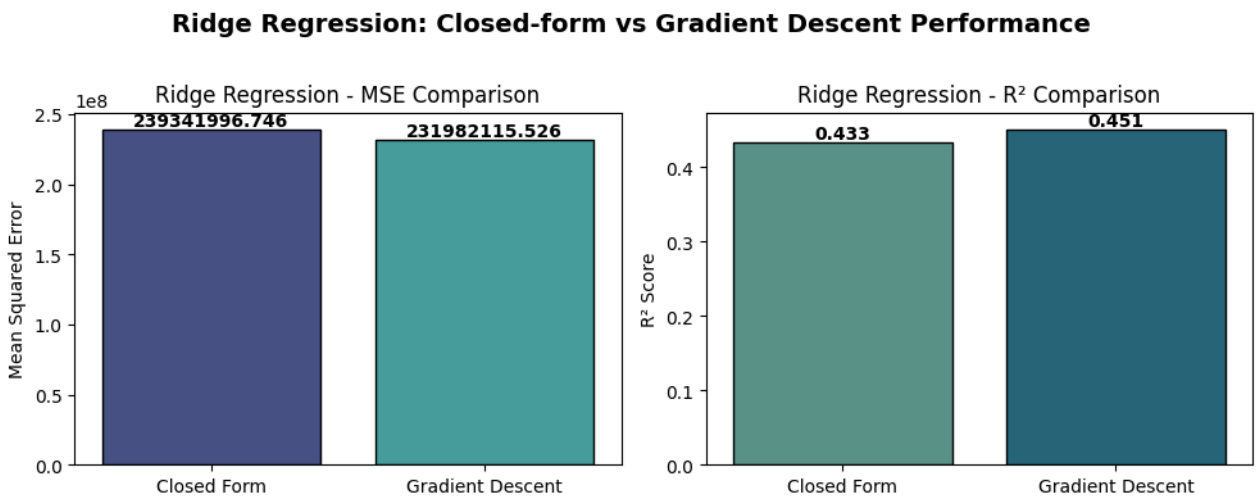
```
plt.figure(figsize=(8,5))
plt.scatter(y_test, y_pred_closed, color='blue', label='Closed-form', alpha=0.7)
plt.scatter(y_test, y_pred_gd, color='red', label='Gradient Descent', alpha=0.7)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'k--')
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.legend()
plt.title("Predicted vs Actual Prices (Closed-form vs Gradient Descent)")
plt.show()
```



Repeat (1), (2), and (3) with l2 regularization

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import mean_squared_error, r2_score

mse_ridge_closed = mean_squared_error(y_test, y_pred_ridge_closed)
mse_ridge_gd = mean_squared_error(y_test, y_pred_ridge_gd)
r2_ridge_closed = r2_score(y_test, y_pred_ridge_closed)
r2_ridge_gd = r2_score(y_test, y_pred_ridge_gd)

models = ['Closed Form', 'Gradient Descent']
mse_values = [mse_ridge_closed, mse_ridge_gd]
r2_values = [r2_ridge_closed, r2_ridge_gd]

plt.figure(figsize=(10,4))

plt.subplot(1, 2, 1)
sns.barplot(x=models, y=mse_values, palette='mako', edgecolor='black')
plt.title('Ridge Regression - MSE Comparison')
plt.ylabel('Mean Squared Error')
for i, v in enumerate(mse_values):
    plt.text(i, v + 0.01 * max(mse_values), f"{v:.3f}", ha='center', fontweight='bold')

plt.subplot(1, 2, 2)
sns.barplot(x=models, y=r2_values, palette='crest', edgecolor='black')
plt.title('Ridge Regression - R² Comparison')
plt.ylabel('R² Score')
for i, v in enumerate(r2_values):
    plt.text(i, v + 0.01 * max(r2_values), f"{v:.3f}", ha='center', fontweight='bold')

plt.suptitle('Ridge Regression: Closed-form vs Gradient Descent Performance', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```

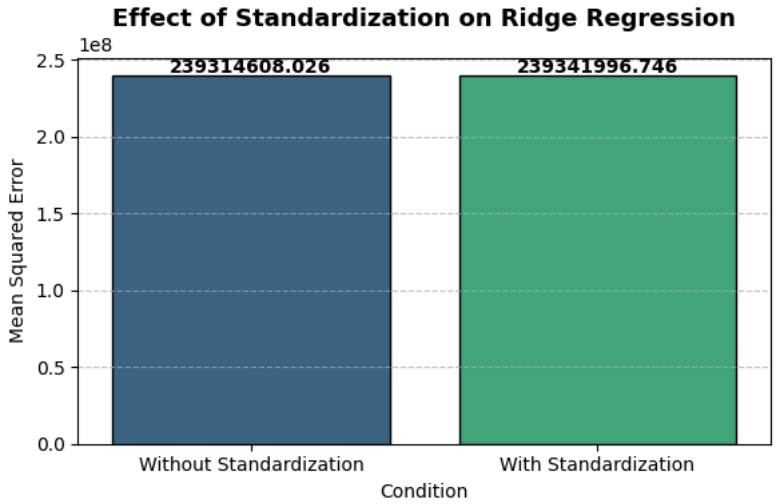
```
C:\Users\Asus Vivobook\AppData\Local\Temp\ipykernel_13696\479695819.py:21: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

  sns.barplot(x=models, y=mse_values, palette='mako', edgecolor='black')
C:\Users\Asus Vivobook\AppData\Local\Temp\ipykernel_13696\479695819.py:29: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

  sns.barplot(x=models, y=r2_values, palette='crest', edgecolor='black')
```



Compare results in (4) with and without data standardization

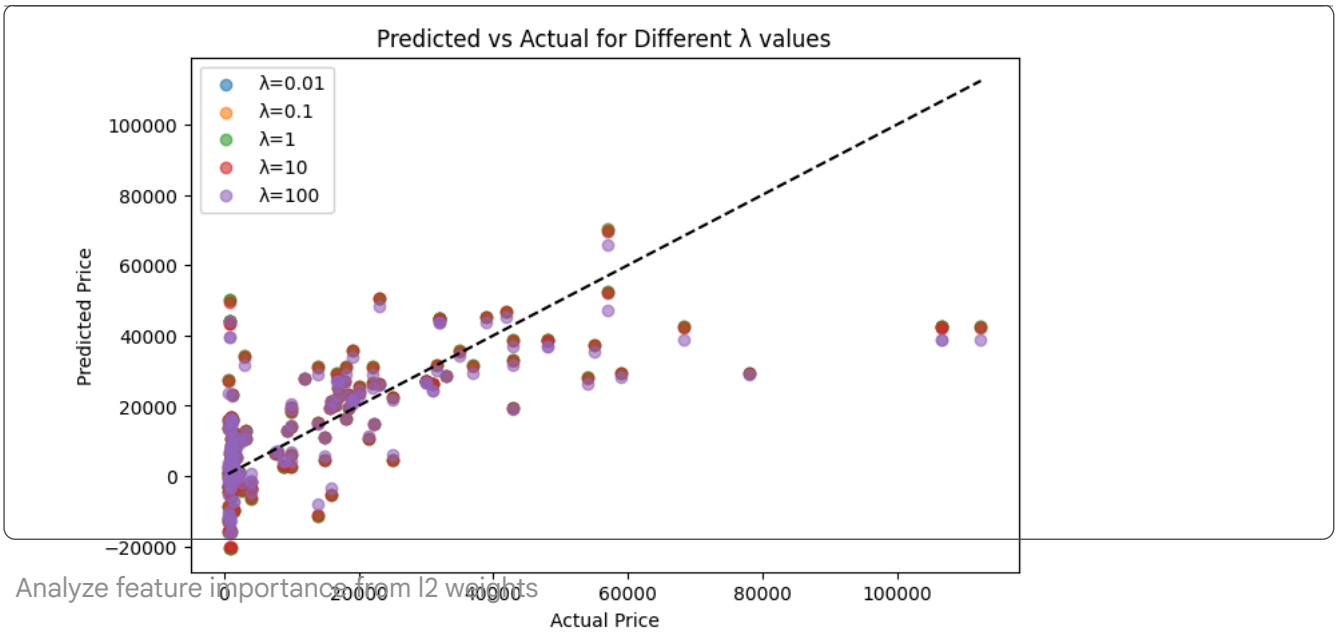
```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import mean_squared_error

mse_without_std = mean_squared_error(y_test_ns, y_pred_ns)
mse_with_std = mean_squared_error(y_test, y_pred_ridge_closed)

conditions = ['Without Standardization', 'With Standardization']
mse_values = [mse_without_std, mse_with_std]

plt.figure(figsize=(6,4))
sns.barplot(x=conditions, y=mse_values, palette='viridis', edgecolor='black')
plt.title('Effect of Standardization on Ridge Regression', fontsize=13, fontweight='bold')
plt.ylabel('Mean Squared Error')
plt.xlabel('Condition')

for i, v in enumerate(mse_values):
    plt.text(i, v + 0.01 * max(mse_values), f"{v:.3f}", ha='center', fontweight='bold')

plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

```
C:\Users\Asus Vivobook\AppData\Local\Temp\ipykernel_13696\1340436231.py:15: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

  sns.barplot(x=conditions, y=mse_values, palette='viridis', edgecolor='black')
```



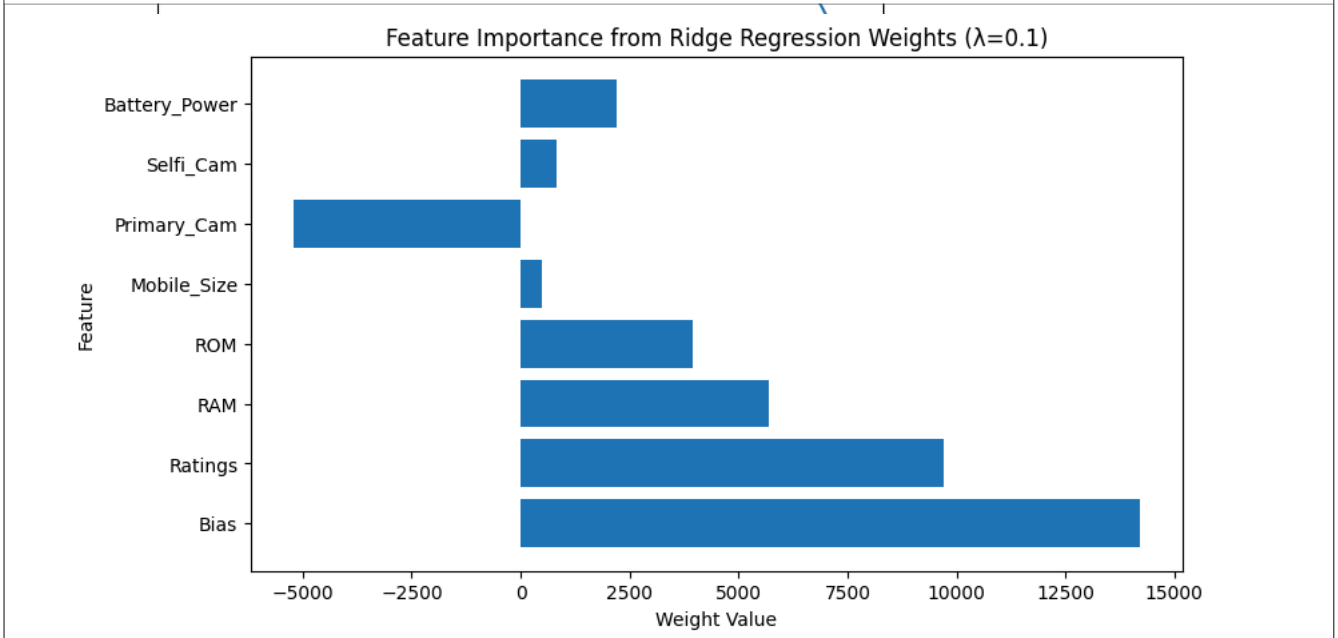
Plot predicted vs. actual on test data for different λ values

```
lambdas = [0.01, 0.1, 1, 10, 100]
mse_values = []

plt.figure(figsize=(8,5))
for lam in lambdas:
    theta = ridge_closed_form(X_train_b, y_train, lam)
    y_pred = X_test_b.dot(theta)
    mse = mean_squared_error(y_test, y_pred)
    mse_values.append(mse)
    plt.scatter(y_test, y_pred, label=f"λ={lam}", alpha=0.6)

plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'k--')
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.legend()
plt.title("Predicted vs Actual for Different λ values")
plt.show()

plt.figure(figsize=(7,4))
plt.plot(lambdas, mse_values, marker='o')
plt.xscale('log')
plt.xlabel("λ (Regularization Strength)")
plt.ylabel("MSE")
plt.title("MSE vs λ")
plt.show()
```



Analyze feature importance from l2 weights

```
feature_names = ['Bias'] + list(data.drop('Price', axis=1).columns)
weights = theta_ridge_closed.flatten()

plt.figure(figsize=(9,5))
plt.barh(feature_names, weights)
plt.title("Feature Importance from Ridge Regression Weights (λ=0.1)")
plt.xlabel("Weight Value")
plt.ylabel("Feature")
plt.show()

print("Weights influence interpretation:")
for name, weight in zip(feature_names, weights):
    print(f"{name:15s} → {weight:.4f}")
```




```
Weights influence interpretation:
Bias            → 14202.1832
Ratings         → 9709.1410
RAM             → 5707.2242
ROM             → 3937.1615
Mobile_Size     → 479.2579
Primary_Cam     → -5216.4926
Selfi_Cam       → 836.0698
Battery_Power   → 2211.5519
```

Bank Note Authentication

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification
from mpl_toolkits.mplot3d import Axes3D
import seaborn as sns
```

```
data = pd.read_csv("BankNote_Authentication.csv")
print("Dataset Loaded Successfully!")
print(data.head())
print("\nDataset Info:\n")
print(data.info())
```

```
Dataset Loaded Successfully!
   variance  skewness  curtosis  entropy  class
0   3.62160    8.6661   -2.8073 -0.44699      0
1   4.54590    8.1674   -2.4586 -1.46210      0
2   3.86600   -2.6383    1.9242  0.10645      0
3   3.45660    9.5228   -4.0112 -3.59440      0
4   0.32924   -4.4552    4.5718 -0.98880      0

Dataset Info:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1372 entries, 0 to 1371
Data columns (total 5 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   variance  1372 non-null   float64
 1   skewness  1372 non-null   float64
 2   curtosis  1372 non-null   float64
 3   entropy   1372 non-null   float64
 4   class     1372 non-null   int64
dtypes: float64(4), int64(1)
memory usage: 53.7 KB
None
```

```
X = data.drop('class', axis=1).values
y = data['class'].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training samples:", X_train.shape[0])
```

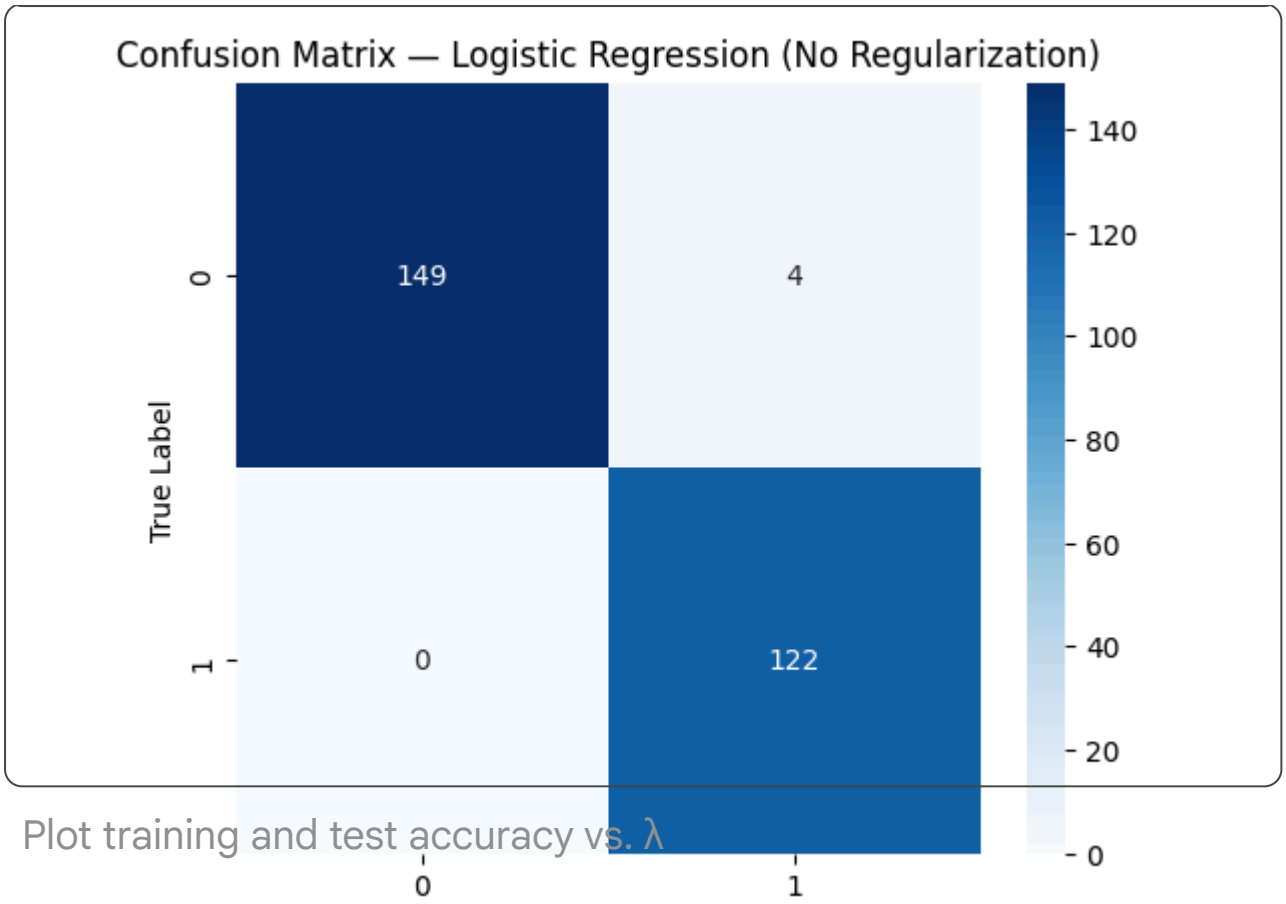
```
print("Testing samples:", X_test.shape[0])
```

```
Training samples: 1097
Testing samples: 275
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

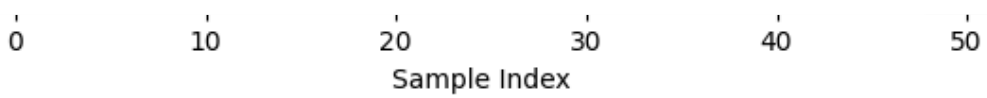
Fit classification model with and without l2 regularization; compare accuracies

```
model_no_reg = LogisticRegression(penalty=None, solver='lbfgs', max_iter=100
model_no_reg.fit(X_train_scaled, y_train)

y_pred_no_reg = model_no_reg.predict(X_test_scaled)

plt.figure(figsize=(6,5))
sns.heatmap(confusion_matrix(y_test, y_pred_no_reg), annot=True, fmt='d', cm
plt.title("Confusion Matrix – Logistic Regression (No Regularization)")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()

plt.figure(figsize=(7,5))
plt.plot(y_test[:50], 'bo-', label='Actual')
plt.plot(y_pred_no_reg[:50], 'go--', label='Predicted')
plt.title("Actual vs Predicted Classes (First 50 Samples)")
plt.xlabel("Sample Index")
plt.ylabel("Class Label")
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(6,4))
sns.countplot(x=y_pred_no_reg, palette='viridis')
plt.title("Predicted Class Distribution (No Regularization)")
plt.xlabel("Predicted Class")
plt.ylabel("Count")
plt.show()
```

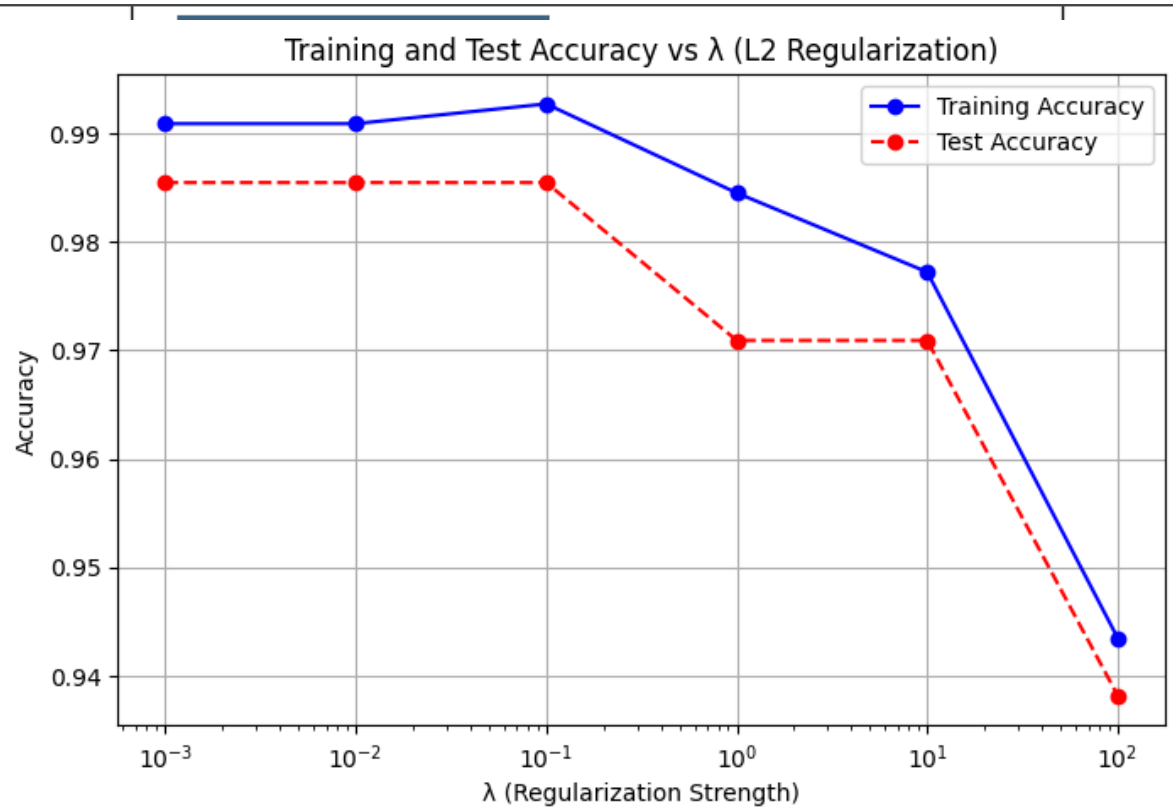


Plot training and test accuracy vs. λ

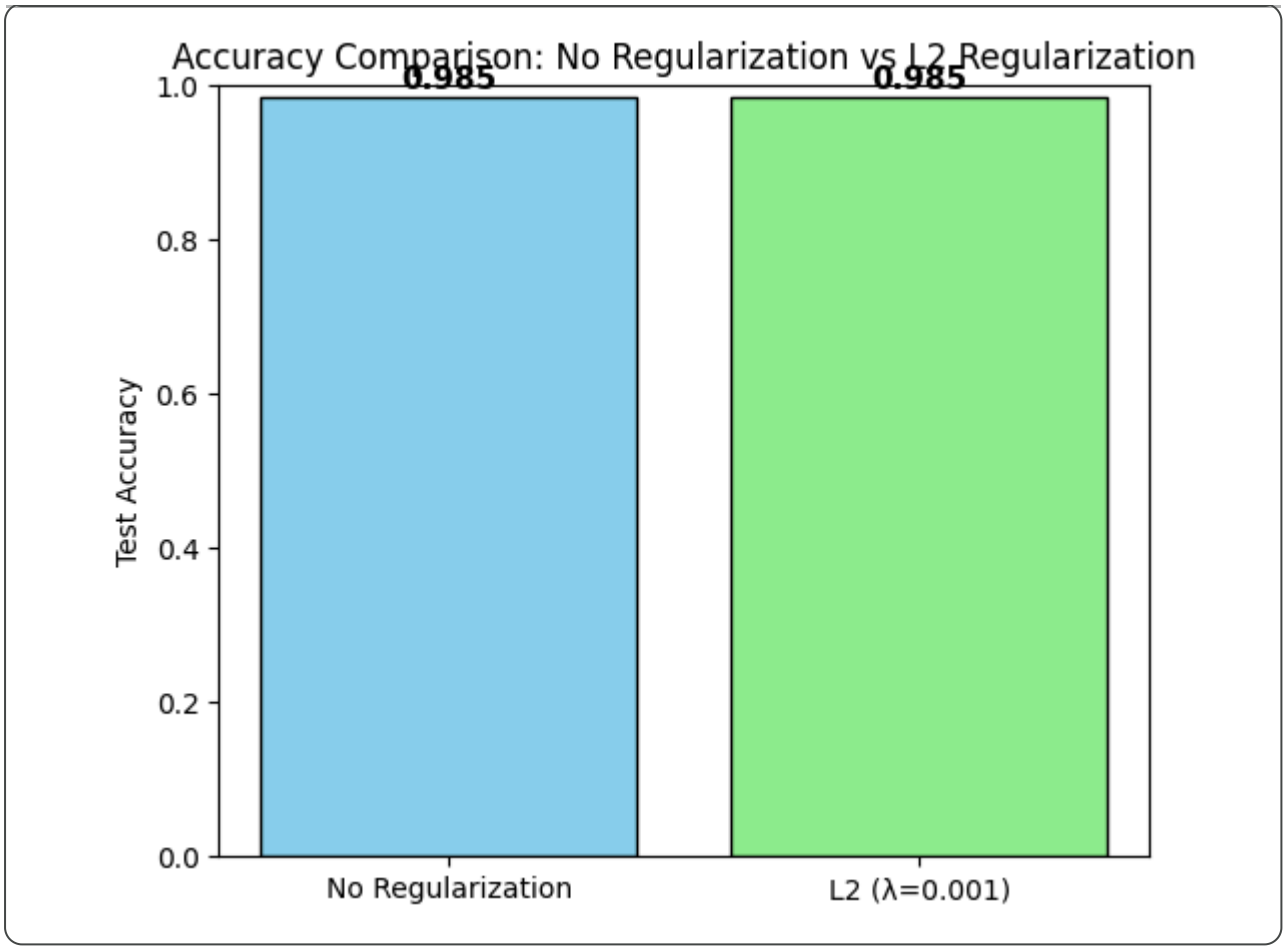
```
lambda_values = [0.001, 0.01, 0.1, 1, 10, 100]
train_acc = []
test_acc = []

for lam in lambda_values:
    clf = LogisticRegression(penalty='l2', C=1/lam, solver='lbfgs', max_iter
    clf.fit(X_train_scaled, y_train)
    train_acc.append(clf.score(X_train_scaled, y_train))
    test_acc.append(clf.score(X_test_scaled, y_test))

plt.figure(figsize=(8,5))
plt.plot(lambda_values, train_acc, 'bo-', label='Training Accuracy')
plt.plot(lambda_values, test_acc, 'ro--', label='Test Accuracy')
plt.xscale('log')
plt.xlabel('λ (Regularization Strength)')
plt.ylabel('Accuracy')
plt.title('Training and Test Accuracy vs λ (L2 Regularization)')
plt.legend()
plt.grid(True)
plt.show()
```




```
C:\Users\Asus Vivobook\AppData\Local\Temp\ipykernel_12552\3300517520.py:24:

Passing `palette` without assigning `hue` is deprecated and will be removed

  sns.countplot(x=y_pred_no_reg, palette='viridis')
```

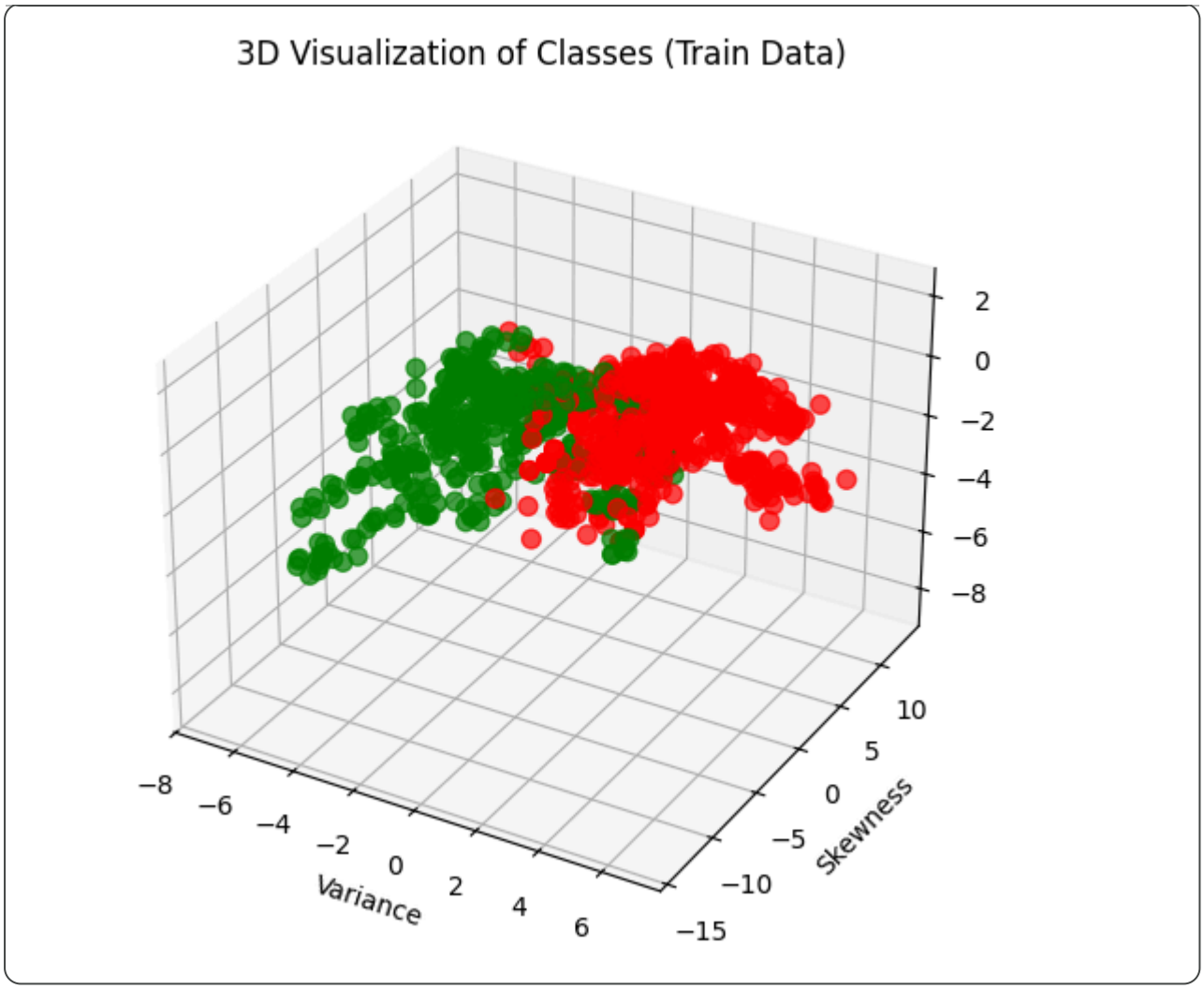



```
best_lambda_idx = np.argmax(test_acc)
best_lambda = lambda_values[best_lambda_idx]
best_acc = test_acc[best_lambda_idx]

# --- Plot Comparison ---
plt.figure(figsize=(6,5))
models = ['No Regularization', f'L2 (λ={best_lambda})']
accuracies = [acc_no_reg, best_acc]
colors = ['skyblue', 'lightgreen']

bars = plt.bar(models, accuracies, color=colors, edgecolor='black')
plt.ylim(0, 1)
plt.title("Accuracy Comparison: No Regularization vs L2 Regularization")
plt.ylabel("Test Accuracy")

# Annotate accuracy values on bars
for bar, acc in zip(bars, accuracies):
    plt.text(bar.get_x() + bar.get_width()/2, acc + 0.01, f"{acc:.3f}",
             ha='center', fontsize=11, fontweight='bold')

plt.show()
```



Visualize 3D plot using three important features

```
fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection='3d')

colors = ['red' if label == 0 else 'green' for label in y_train]
ax.scatter(
    X_train[:,0], X_train[:,1], X_train[:,3],
    c=colors, alpha=0.7, s=50
)
ax.set_xlabel('Variance')
ax.set_ylabel('Skewness')
ax.set_zlabel('Entropy')
ax.set_title('3D Visualization of Classes (Train Data)')
plt.show()
```



Intentionally introduce outliers by shifting data points

```
X_outlier = X_train_scaled.copy()
np.random.seed(42)
outlier_indices = np.random.choice(range(len(X_outlier)), size=10, replace=F
X_outlier[outlier_indices] += np.random.normal(5, 2, X_outlier[outlier_indic

print(" Outliers added at indices:", outlier_indices[:5])
```

```
 Outliers added at indices: [ 44 568  56 636 486]
```

Fit classifier on outlier-injected data and comment on its impact

```
clf_outlier = LogisticRegression(penalty='l2', C=1/best_lambda, solver='lbfg
clf_outlier.fit(X_outlier, y_train)

train_acc_outlier = clf_outlier.score(X_outlier, y_train)
test_acc_outlier = clf_outlier.score(X_test_scaled, y_test)

print(" Impact of Outliers:")
```

```
print("Train Accuracy (with outliers):", train_acc_outlier)
print("Test Accuracy (with outliers):", test_acc_outlier)
print("→ Compare to accuracy without outliers:", best_acc)
```

```
 Impact of Outliers:
Train Accuracy (with outliers): 0.9389243391066545
Test Accuracy (with outliers): 0.9272727272727272
→ Compare to accuracy without outliers: 0.9854545454545455
```

```
print("SUMMARY OF OBSERVATIONS")
print(f"Base Accuracy (No Reg): {acc_no_reg:.4f}")
print(f"Best L2 Accuracy (λ={best_lambda}): {best_acc:.4f}")
print(f"Accuracy After Outliers: {test_acc_outlier:.4f}")
print("\n➞ L2 Regularization improved model stability and handled multicollir
print("➞ Outliers reduced model performance — Logistic Regression is sensiti
print("➞ Data standardization ensured faster and stable convergence.")
```

```
SUMMARY OF OBSERVATIONS
Base Accuracy (No Reg): 0.9855
```